SI 507 Final Project

Dataset:

Since the requirement for this final project is the implementation of graph, I chose to use the SNAP Network datasets and the link to the dataset is as below.

The data contains lots of combination of 5 files, which are circles, edges, feat, featname, and egofeat. Each 5-file combination represent an ego (a social center or social community that is centered to one single user). And below is the explanation of each file and how the text file looks like.

https://snap.stanford.edu/data/egonets-Facebook.html

The dataset contains several text files.

1. ***.circles: friend groups (communities)**

   circle0 71 215 54 61 298 229 81 253 193 97 264 29 132

   circle1 173

   above is the sample of circles file, and the meaning is that in circle 0, there are friends 71, 215, 54, etc. This file provides the community labels.

2. ***.edges: Friendship connections among users**

   236 186

   122 285

   Each number in the text file represent a node ID, which is a Facebook user. A row like 236, 186 means that user 236 is a friend with user 186. This represent a undirected edges and this is my main graph comes from.

3. ***.feat: User feature vector**

   1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
   0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
   0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
   0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

   Above is one row in the text file. The first number is the user ID, and the rest are all 0 and 1, each representing one feature. The features are described in featname.txt file. So, let's say this user has 1 in the 27^{th} feature, and such feature represent the master degree. That means such user on Facebook owns a master degree.

4. *.featnames: Dictonaries of what each feature means

   0 birthday;anonymized feature 0

   1 birthday;anonymized feature 1

This file maps feature index → feature name (meaning)

5. *.egofeat: Features of the ego user (the center)

0 0 0 0 0 0 0 0 1 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 1 1 1 1 0 0 0
0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0 0 0 1 0 0 0 0 1 0 0 0 1 0 0 1 0 0 1 0 1 0 0 0 0 0 0 0 1
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 1 0 0 0 0 0 0 0 1 1 0 1 0 1 0 1 0 0 0 0 0 1 0 0 0 0

This is the feature vector for the ego user itself.

Implementation:

Since this dataset is quite large that contains over 4000 nodes with over 88000 edges, I will focus on implementation of graph / network analysis using networkx module instead of writing graph from scratch. However, I will still write my graph code so that I can have the decent practice for future interviews and the algorithms such as BFS and DFS.

Also, there is python wrapper for such C++-based SNAP that is called snap and can be imported through pip. Such graph analysis engine was also provided in the dataset website. But since it works much alike like networkx, I will use networkx instead.

First, I will try to load the ego 0 to the graph first.

Since there are 5 distinct files for each ego center, I made 5 functions to read those files, and a function to build those features into the graph. Detail code is shown in below screenshots.

```python
# 1. load edges
def load_edges(path):
    '''
    loads the edge list from the .edges file
    each row looks like:
    236 186
    122 285
    meaning user 236 is friend with user 186

    create an undirected graph becuz friendships have no direction
    '''
    G = nx.Graph()

    with open(path, 'r') as f:

        for line in f:
            u, v = line.strip().split()

            # convert string -> integer
            u = int(u)
            v = int(v)

            # add an undirected edge to the graph
            G.add_edge(u, v)

    return G
```

```python
# 2. load feature names
def load_featnames(path):
    '''
    loads the feature names from the .featname file
    each row looks like:
    0 Birthday: Born in January
    1 Birthday: Born in February
    followed by a column index

    return a dictionary mapping:
    index -> feature_name
    '''
    featnames = {}
    feature_order = []

    with open(path, 'r') as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            # split on first whitespace ONLY
            parts = line.split(None, 1)

            # must have: [index, feature name]
            if len(parts) != 2:
                print("Skipping malformed line:", line)
                continue

            idx_str, name_str = parts

            try:
                idx = int(idx_str)
            except ValueError:
                print("Skipping line with non-integer index:", line)
                continue

            featnames[idx] = name_str.strip()
            feature_order.append(idx)

    return featnames, feature_order
```

```python
# 3. load features for each node
def load_features(path):
    '''
    loads the node features from the .feat file
    each row looks like:
    1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
    while the first integer is the user ID and the other integers
    are feature vector

    return a dictionary: node_id -> feature_vector(list of integers)
    '''
    feature = {}

    with open(path, 'r') as f:
        for line in f:
            # split only one time and leave the rest to the list
            temp = line.strip().split()
            idx = int(temp[0])
            vectors = list(map(int, temp[1:]))
            feature[int(idx)] = vectors

    return feature
```

```python
# 4. load the ego's own features
def load_egofeat(path):
    '''
    loads the ego user's feature vector
    ego is the central person of this ego-network
    text file looks like this: (like feat file)
    (no user ID included)
    0 0 0 0 0 0 0 0 0 1 0 0 0 0

    returns a list containing only vectors
    '''
    with open(path, 'r') as f:
        temp = f.readline().strip().split()
        vector = list(map(int, temp))

    return vector
```

```python
# 5. load social circles
def load_circles(path):
    '''
    loads circle (community) friendship from .circles file
    the text file looks like:
    circle0 71  215
    circle1 173

    returns a dictionary with key
    dict: "circle0" -> [71, 215]
    '''
    circle = {}

    with open(path, 'r') as f:
        for line in f:
            temp = line.strip().split()
            key = temp[0]
            value = list(map(int, temp[1:]))

            circle[key] = value

    return circle
```

```python
# 6. build the full graph object
def build_graph(ego_id):
    '''
    this function loads all 5 files for a given ego
    then constructs a fully annotated networkx graph
    '''
    # define file paths by changing the ego_id
    edges_fileName = f"{ego_id}.edges"
    featname_fileName = f"{ego_id}.featnames"
    feat_fileName = f"{ego_id}.feat"
    egofeat_fileName = f"{ego_id}.egofeat"
    circle_fileName = f"{ego_id}.circles"

    # get the current working directory
    script_file = os.path.abspath(__file__)
    script_dir = os.path.dirname(script_file)

    # get the files path
    edges_path = os.path.join(script_dir, "data", edges_fileName)
    featname_path = os.path.join(script_dir, "data", feat_fileName)
    feat_path = os.path.join(script_dir, "data", feat_fileName)
    egofeat_path = os.path.join(script_dir, "data", egofeat_fileName)
    circle_path = os.path.join(script_dir, "data", circle_fileName)
```

```python
    # load data
    print("Loading edges...")
    G = load_edges(edges_path)

    print("Loading feature names...")
    featnames = load_featnames(featname_path)

    print("Loading node features...")
    features = load_features(feat_path)

    print("Loading ego features...")
    ego_features = load_egofeat(egofeat_path)

    print("Loading circles...")
    circles = load_circles(circle_path)

    print("Attaching features to nodes")
    for node, feat_vector in features.items():
        if node not in G:
            G.add_node(node)
        G.nodes[node]["feature"] = feat_vector
```

```python
    print("Attaching circle labels")
    for circle_name, members in circles.items():
        for node in members:
            # if member node in circle in current graph
            if node in G.nodes:
                # the node in the graph would get a circles key and its value would
                # be a list that get the circle name
                G.nodes[node].setdefault("circles", []).append(circle_name)

    # create an explicit ego node
    ego_node_id = f"ego_{ego_id}"
    G.add_node(ego_node_id)
    G.nodes[ego_node_id]["feature"] = ego_features
    G.nodes[ego_node_id]["is_ego"] = True

    print("Graph loaded completely")
    return G, featnames, features, ego_features, circles, ego_node_id
```

After running the build_graph function, we obtain all essential components of the network, including the graph object, the feature-name dictionary, the feature order, the node features, the ego user's features, the ground-truth circles, and the ego user ID. These outputs allow us to apply additional functions to further explore and interact with the graph.

Interaction:

I implemented two visualization functions: one that displays the graph with the **true circles** from the dataset and another that plots the same graph using **Louvain community detection**. Comparing the two visualizations, we observe that Louvain identifies far fewer communities—about **8 to 9**—compared to the dataset's **22 to 23 true circles** for ego 0.

In addition to visualization, users can query any node to inspect its properties, including its true circle membership, its Louvain community assignment, its feature vector, and its degree. Even if users choose not to display the graph, they can still query nodes to obtain their IDs and corresponding information. Finally, the system supports simple ID lookup: users can input any user ID to check whether that node exists in the graph.

Below is the UI of my program:

(introduction of the program)

=====Network Analysis=====

The dataset the programmer used is from SNAP (Facebook Network)

(loading the dataset automatically)

Now Loading the Graph / Network...

Loading edges...

Loading feature names...

Loading node features...

Loading ego features...

Loading circles...

Attaching features to nodes

Attaching circle labels

Graph loaded completely

Finished!

(visualization prompt with true circle)

Do you want to see the Graph with true circles? (y/n) y

(visualization prompt with Louvain community)

Do you want to see the Graph with louvain communities? (y/n) y

(prompt the user which circle he/she want to see, and will output the features wthin that circle)

Check the community / circle feature name summary (louv / cir / n for quit): cir

Input circle id: 3

[('Locale: Asia', '93.3%'), ('Gender: Female', '80.0%'), ('Education Type: Public School', '76.7%'), ('School Attended: Community College A', '60.0%'), ('Education Type: International School', '50.0%'), ('Speaks Language: Chinese', '36.7%'), ('Speaks Language: Portuguese', '30.0%'), ('Graduation Year: 2007', '26.7%'), ('Graduation Year: 2013', '26.7%'), ('Lives In: Philadelphia', '26.7%')]

(show some sample nodes in the graph for users)

Here are some nodes in the Graph

[236, 186, 122, 285, 24]

(prompt user to see which node user want to check)

Which node do you want to query (n for quit): 236

Node: 236

Degree: 37

True Community: ['circle4', 'circle15']

Louvain Community: 0

Active Features:

- Birthday Month: August

- Major/Focus: Engineering

- School Attended: Community College A

- Education Type: Public School

- Graduation Year: 2007

- Gender: Male

- Locale: Asia

- Lives In: Philadelphia

- Works At: Company K

- Works At: Company N

- Works At: Company V

- Work End Year: 2018

- Work Location: Chicago

- Work Location: New York

- Work Location: Washington DC

- Job Position: Manager

- Job Position: Supervisor

- Work Start Year: 2019

- Work Team: Team A

- Work Team: Team D

- Work Team: Team E

- Work Team: Team G

(prompt user if he/she want to check again)

Do you want to query again? (y/n) y

Which node do you want to query (n for quit): 122

Node: 122

Degree: 63

True Community: ['circle4', 'circle15']

Louvain Community: 0

Active Features:

- School Attended: Community College A

- Education Type: Public School

- Graduation Year: 2007

- Gender: Male

- Locale: Asia

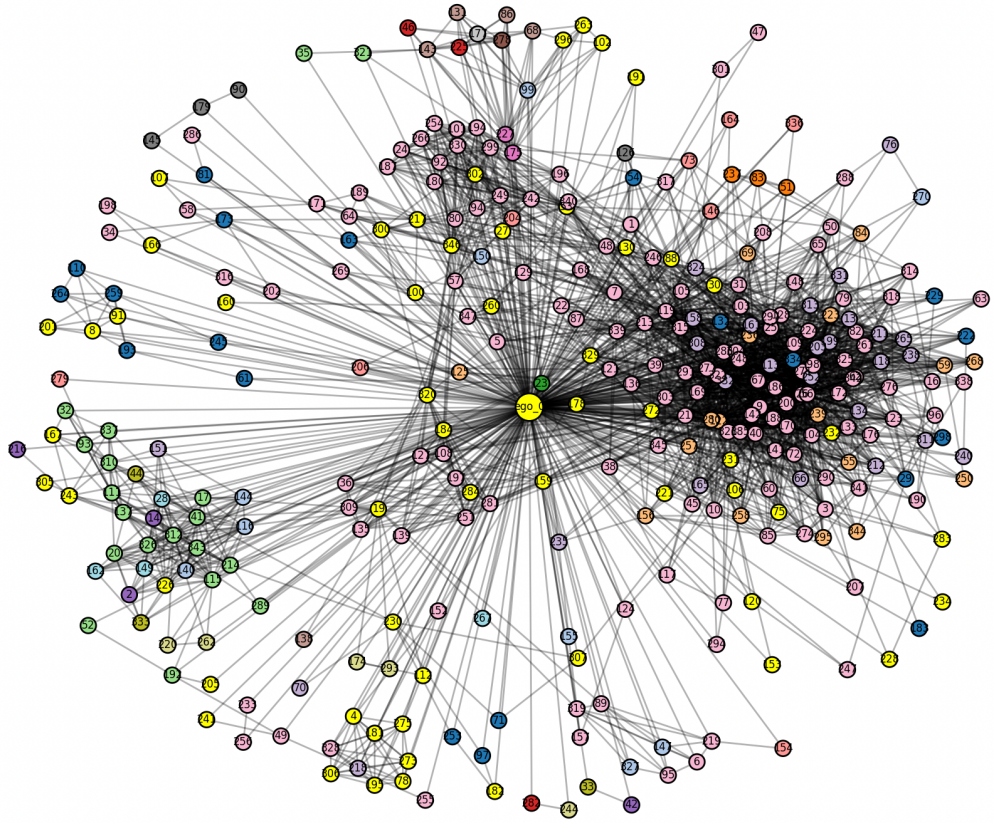
- Works At: Company G

Do you want to query again? (y/n) n

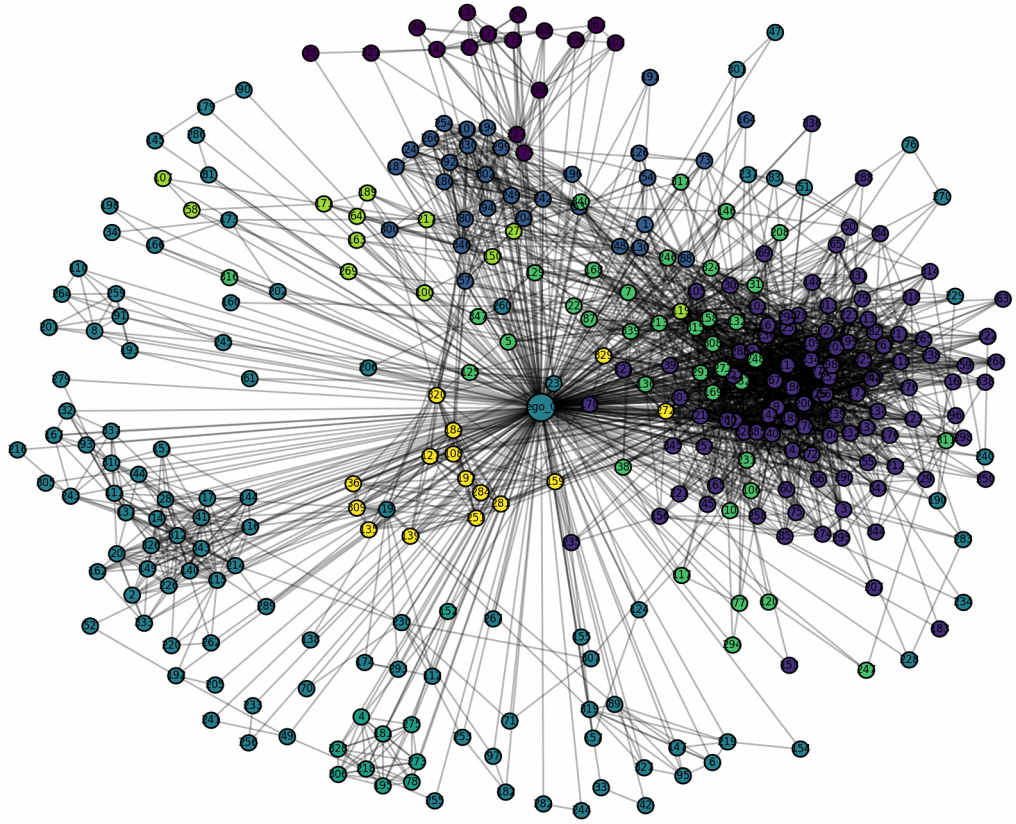
(ending)

Thank you for trying the network analysis!

Visualization of Ego Network 0



Visualization of Ego Network 0 with Louvain Community



Conclusion:

In this project, I worked with a SNAP dataset created by Stanford University. Although the dataset is real, the feature names are anonymized for confidentiality, making it difficult to interpret how specific features relate to user circles or communities. To gain more insight, I used an LLM to rewrite the featname file into a more interpretable format.

The dataset includes many ego networks, each independent from the others. For this project, I focused on ego ID 0, although the program is fully capable of handling any other ego network. Through this project, I learned how to use NetworkX, explored graph and network analysis concepts, and applied the Louvain community detection algorithm.